

A High-Throughput and Flexible Architecture Based on a Reconfigurable Mixed-Radix FFT With Twiddle Factor Compression and Conflict-Free Access

Chen Yang^{ID}, Member, IEEE, Junfeng Wu, Siwei Xiang, Liyan Liang, and Li Geng^{ID}

Abstract—Mixed-radix fast Fourier transform (FFT) algorithms are widely adopted in high-performance communication systems, such as 5G systems. However, in the decimation-in-time (DIT) FFT algorithm, neither the input time-domain series nor the intermediate-stage data are fetched in natural order, which causes a mismatch between the processing element (PE) computation speed and the memory bandwidth. In addition, multiple duplicate interstage twiddle factor (TF) generation units are designed to provide interstage TFs in parallel, and such duplication results in considerable waste of computing and storage resources. In this article, we propose a flexible and reconfigurable architecture based on a mixed-radix FFT approach, supporting 61 different FFT sizes modes from 12 to 3240 points (2^a or $12 \times n$, $n \leq 270$, and $n = 2^a 3^b 5^c$). A DIT-FFT-based multiple parallel changeable-radix butterfly unit (BU) is designed to improve hardware resource utilization. Around the PE array, a conflict-free access structure with a hardware-friendly address generating method is presented. In addition, a TF sharing and compression structure is designed to reduce the area and delay of the TF generation units. The FFT architecture is implemented in semiconductor manufacturing international corporation (SMIC) 40-nm CMOS technology with a working frequency of 483 MHz, performing at 654 MS/s for 2048-point and 1536-point FFTs. Compared to the state-of-the-art mixed-radix FFT designs, our architecture achieves improvements of up to 5.89× in throughput and area efficiency and supports more FFT modes.

Index Terms—Conflict-free, dynamically reconfigurable, fast Fourier transform (FFT), mixed-radix, twiddle factor (TF) compression.

I. INTRODUCTION

THE fast Fourier transform (FFT), as a fast implementation of the discrete Fourier transform (DFT), has become one of the most important classes of algorithms in signal processing. FFT algorithms are widely used in communication systems based on the orthogonal frequency-division

multiplexing (OFDM) transmission mode, such as 802.11n/ac, 4G LTE/long-term evolution-advanced (LTE-A), and 5G systems [1]. FFTs can be divided into two types: power-of-two FFTs and non-power-of-two FFTs. 4G LTE/LTE-A and 5G communications require FFT sizes ranging from 12 to 2400. To support the high data rate of 4G/5G technology, non-power-of-two FFT algorithm is not sufficient for current OFDM requirements.

Many different FFT architectures have been proposed for processing power-of-two FFTs based on radix- 2^k units [3], [4], [5]. However, the performance of those processors is limited for non-power-of-two FFTs. To achieve satisfactory FFT performance for 5G communication, radix-3 [6], [7], [8], [9] and radix-5 [10], [11] units have been added to radix- 2^k FFT systems. However, their throughput rate (TR) is restricted to $1 \times$ the clock rate for a group of FFT data [2] due to the single-path delay feedback (SDF) architecture. In contrast to an SDF FFT implementation, a memory-based FFT implementation supports multiple data paths, providing high throughput for calculation. For the memory-based FFT approach, the conflict-free access method plays a prominent role. A scheduling scheme for data access for a memory-based FFT processor using radix- 2^k units [12] has been proposed, eliminating memory access conflicts and achieving high throughput and a high speed-area ratio (SAR).

Regarding efforts to achieve both high performance and high flexibility simultaneously, three issues arise in mixed-radix FFT implementations. First, the diversity type of computing units needed for different radices results in low hardware resource utilization. Such computing units are referred to as butterfly units (BUs) in FFT implementations. Yu and Yen [8] designed two kinds of BUs: one for power-of-two FFTs and another for the radix-3 FFT. However, when the processor is calculating a 2^k -point FFT, the BUs for the radix-3 FFT will be in the idle state. An effective strategy is to adopt coarse-grained reconfigurable architectures (CGRAs). In [13], radix-2/3/4/5 BUs were achieved in one reconfigurable processing element (PE), and in [14] and [15], radix-7 capabilities were added to the reconfigurable PE. However, such a reconfigurable PE essentially relies on adjusting radix-7 or radix-5 BUs to suit other radices. A single modified radix-5 BU can only be reconfigured as one radix-2, radix-3, or radix-4 unit. Therefore, such a reconfigurable PE array remains limited in terms of the achievable improvement in hardware resource utilization. Second, the FFT data processing procedure is

Manuscript received 9 February 2023; revised 28 May 2023; accepted 8 July 2023. Date of publication 8 August 2023; date of current version 27 September 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62176206, in part by the Aeronautical Science Foundation of China under Grant 2020Z066070001, and in part by the Key-Area Research and Development Program of Guangdong Province under Grant 2019B010154002. (Corresponding author: Chen Yang.)

The authors are with the School of Microelectronics, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China (e-mail: chyang00@xjtu.edu.cn; wjf2016@stu.xjtu.edu.cn; X1004312107@stu.xjtu.edu.cn; rockauthor214@stu.xjtu.edu.cn; gengli@xjtu.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TVLSI.2023.3298943>.

Digital Object Identifier 10.1109/TVLSI.2023.3298943

1063-8210 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

not performed in natural order, resulting in memory access conflicts in the data memory. Multiple strategies have been proposed with the aim of mitigating memory access conflicts in the intermediate stages of FFT computations. Most of them reduce memory access conflicts by arranging interrelated data in different banks. Once the number of FFT points has been determined, the data locations in memory will be fixed until the end of the calculation, as in [2]. However, with an increasing number of FFT stages, memory access conflicts will still arise. Third, multiple duplicate twiddle factor (TF) generation units are designed to provide TFs for highly parallel data processing in a PE array. There are two methods of generating the values of the interstage TFs: calculating the values based on the coordinate rotation digital computer (CORDIC) algorithm [26], [27] and using memory-based TF units [2], [11], [33]. However, neither the CORDIC- nor memory-based method can avoid redundancies in hardware or storage.

This article presents the flexible conflict-free memory-based FFT (CFMBT) architecture, achieving high throughput and area efficiency. The novelty of this article is embodied in three aspects: reconfigurable PE array, dynamic data memory access strategy (DDMAS), and buffer design, low-redundancy TF generation unit. To reach high performance, an extensible PE array is designed to perform butterfly computations in parallel. Furthermore, to match the computational parallelism, a DDMAS is proposed that can completely eliminate memory access conflicts in the data memory. To improve area efficiency, the key idea behind this article is to present a sharing and compression structure employing a variety of optimization strategies, thereby reducing the storage redundancy for TFs. To achieve high flexibility, this article proposes a dynamically reconfigurable PE array that can be transformed into radix-2/3/4/5 BUs. In summary, the key contributions of the proposed architecture are as follows.

- 1) A unified PE array is designed based on the decimation-in-time (DIT) FFT algorithm to calculate different BUs in radix-2/3/4/5 FFTs. Multiple PEs can be combined through reconfigurable crossbar interconnections to form a complex BU. Compared with customized BU design which can only support one kind of radix, our proposed unified PE array can reuse the same hardware resources. Therefore, different BUs in different FFT stages can be executed on reconfigurable homogeneous PE.
- 2) A DDMAS is designed according to our proposed bit reordering schedule. In each stage of the FFT, the locations of the data in memory are rearranged. Besides, DDMAS is in accordance with a bit reordering schedule. Through this dynamic access method, memory access conflicts can be avoided. In this way, data are supplied at a rate that can match the performance of the PE array. In DDMAS, the banks in the data memory are divided into several groups, and the computation order for the data is adjusted as well. Thus, the data can be read or written sequentially in an inter- or intra-group manner, respectively. As a result, the address generator for DDMAS is simple.
- 3) A novel strategy is proposed for the sharing and compression of the TFs. Through rearrangement of the computation sequence of the input series in one stage, multiple BUs can reuse the same TF. Hence, the storage overhead and the number of calculation modules for the TFs can be reduced. We also proposed a modified TF

partitioning strategy and a conflict-free prime TF access method. In this way, the storage redundancy can be further reduced.

- 4) The CFMBT architecture supports 61 different FFT sizes modes from 12 to 3240 points which are broader than state-of-the-art works. By means of multiple optimization strategies, CFMBT achieves high throughput and flexibility. Our design has been synthesized in 40-nm technology. Its performance has been fairly compared with the results reported for the state-of-the-art CGRAs, including power-of-two FFT and mixed-radix FFT architectures. Where necessary, the reported results have been scaled to the number of logic gates or computation units for fair comparisons.

II. RELATED WORK AND MOTIVATION

A. Related Work

A number of studies have proposed various FFT hardware architectures. Using the dynamically reconfigurable resource array (DRRA) fabric [16], a size-configurable radix-2 FFT processor has been achieved by combining local register files with memory and applying a generic FFT address generation algorithm and scheme for mapping an arbitrary-size FFT. Similar work on radix-2 FFT implementations was performed in [17], [18], and [19]. Kim et al. [20] achieved an area-efficient FFT/inverse FFT processor by applying a shared multiplier scheduling scheme (SMSS). Other works have focused on expanding the radix-2 FFT to radix- 2^k [12], [21], [22], [23], designing conflict-free access strategies [24], [25] and decreasing the area of the TF generation units [26], [27]. However, these works support only 2^k -point FFTs. Sheng-Yeng et al. [7], Yu and Yen [8], Shih et al. [9], and Shih et al. [31] added the radix-3 or radix-5 FFT to their system based on an SDF FFT implementation. Xia et al. [28] went further, designing two individual circuit cores. One core was for 2^k -point FFTs, and the other was for non-power-of-two radix-3/4/5-based FFTs. More FFT scales were supported in this design; however, this two-core design resulted in hardware waste and high area costs. Meanwhile, its data memory consumption was $2N + 3N$, and it was implemented with dual-port memory, resulting in a large memory scale. The designers of an SDF-based FFT chip [11] achieved mixed-radix (non-power-of-two) FFTs by improving the BUs, simplifying the data path and applying a memory-based TF unit. However, these works implementing SDF method did not achieve a high TR for mixed-radix FFTs. A memory-based processor [2] achieved high FFT throughput by organizing multiple BUs and optimizing memory access conflicts. In contrast to SDF-based implementations, the TR of memory-based FFT implementations is not limited to $1 \times$ the clock rate, and thus, such implementations are more suitable for the high speeds required for long-term evolution (LTE) communication technology. However, the static data location scheme of conventional memory-based implementations will lead to a mismatch between the PEs' TR and the memory bandwidth of the data. Wang et al. [12], Jui et al. [29], and Luo et al. [30] optimized the memory access scheme and achieved conflict-free data access for radix- 2^k FFTs. However, these methods are limited in their ability to meet the needs of mixed-radix FFTs.

B. Motivation

First, a mixed-radix FFT algorithm with high throughput and low area consumption is required for 5G

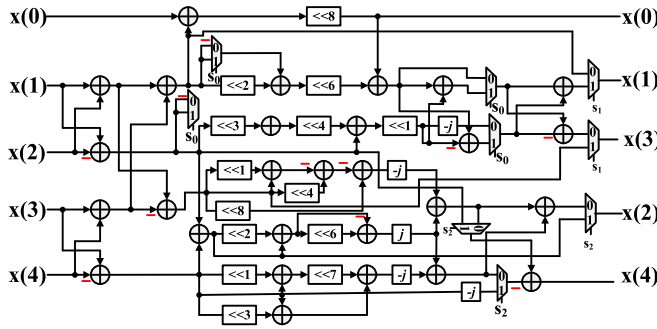


Fig. 1. Reconfigurable architecture for radix-2/3/4/5 FFTs [13].

communication. For example, FFT sizes ranging from 12 points to 1296 points are required in LTE [7]. For 2^k -point FFTs, many high-speed FFT implementations have been achieved [17], [18], [19], [22], [23], [24], [25]. However, the applicability of 2^k -point FFTs is limited in OFDM systems. SDF method restricts the TR to $1 \times$ the clock rate. Considering that 5G communication technology requires a high data rate, a memory-based FFT implementation is a better choice. Qureshi et al. [13] proposed a reconfigurable BU supporting radix-2/3/4/5 FFTs, as shown in Fig. 1. However, the hardware utilization is not high enough in this architecture, when such a reconfigurable BU is running at a lower radix, such as radix-2 or radix-3, only part of the data path will be utilized.

Second, the data memory access bottleneck is another factor that influences the FFT speed. Data are read from one data memory and transferred to the PE array. Then, after the calculations of the PE array, the operation result is written to another data memory. In memory-based FFT implementations, the PE array's TR is increased by increasing the parallelization of the PEs. As a result, a multibank memory structure with conflict-free data access is a key factor affecting the speed of the FFT core. According to the roofline model [32], the FFT TR is related to the calculation speed and the access speed. In memory-based FFT implementations, computational resources are always abundant. Therefore, memory access conflicts are the main concern. Wang et al. [12], Jui et al. [29], and Luo et al. [30] used single-port memory to achieve conflict-free access for radix- 2^k FFTs; however, this method is difficult to implement for mixed-radix FFTs such as radix-3 or radix-5. Liu and Liu [2] proposed a dislocation scheme for radix-2/3/5; by shifting the locations of the data, conflicts can be reduced. However, with increasing data size and an increasing number of FFT stages, the problem of memory access conflicts becomes more serious. For example, the PE unit proposed in [2] supports four radix-3/4 computations in parallel. However, due to memory limitations, the FFT core can compute only three radix-4 butterflies simultaneously, as shown in Fig. 2. This problem is related to the fact that in different FFT stages, the scheduling order of the data is different. If the locations of the data are fixed, then conflicts are difficult to avoid with an increasing number of stages.

The TFs are necessary parameters of the FFT. As such, the area occupied by the TF generation units and the TR of these units affect the performance of the FFT core. A general method is to apply multiple duplicate TF units for each BU to ensure that its TR matches the PE array's demand. However, duplicate TF units incur hardware redundancy. As the PE array performance increases, this problem will become more serious. There are three common methods (i.e., online

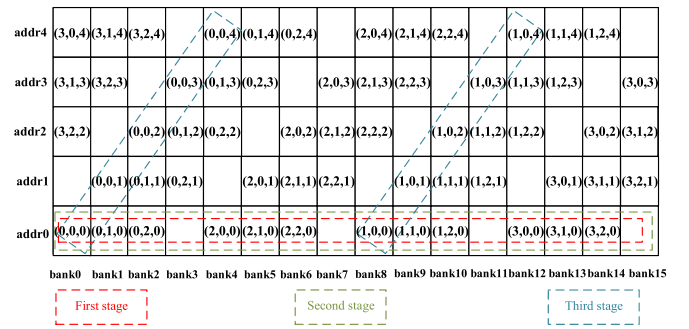


Fig. 2. Data allocation strategy (n_1 , n_2 , and n_3) and index mapping (binary bit field) for a 60-point FFT ($N_1 = 4$, $N_2 = 3$, $N_3 = 5$) in [2].

TABLE I
COMPARISON OF DIFFERENT TF UNITS

Work	Method	Accuracy	Wait Time	Area Saving ^{*1}
[2], [26]	Online Calculation	precise	longest (≥ 7 clk)	41.3%、 20.2%-33.0%
[10], [11]	Single-Table Approximation	approximate	medium (≥ 1 clk)	64.72%、 82.10% - 84.13%
[9]	ROM partitioning	precise	shortest (usually 1 clk)	53.0%

*1 Area Saving is compared with the area before using optimization scheme

calculation, single-table approximation, and read-only memory (ROM) partitioning) of generating TF values, as summarized in Table I. In the online calculation method [2], [26], [27], the value is computed online using the CORDIC algorithm. However, online calculation will necessitate iterative calculations and result in a long TF generation latency. In addition, its area savings are limited compared with those of the other two methods, as shown in Table I. The single-table approximation method (STAM) used in [10] and [11] leads to non-negligible errors in a communication system. The third method is based on a ROM partitioning strategy, which is an improved variant of the memory-based method. Lin et al. [33] applied such method and stored only TFs located within one-eighth of the unit circle. Shih et al. [9] developed an area-reduced scheme by applying the ROM partitioning strategy. It combined with the memory-based strategy and online calculation achieves the lowest TF generation latency. Thus, it is possible to compensate for this disadvantage of ROM partitioning through a TF sharing strategy. However, its area consumption is larger than that of the STAM.

To solve the above three issues on hardware flexibility and area efficiency, this article proposed a high-performance BU, conflict-free memory access, and low hardware overhead TF generation unit. First, this article designs a reconfigurable BU for radix-2/3/4/5 computations based on the DIT FFT algorithm. By fully extracting the common operation units among radix-2/3/4/5 FFTs, we designed a general part structure, used as a PE unit. By modifying the interconnections between these homogeneous PEs' array, different kinds of BUs can be implemented. Second, this article develops a dynamic data allocation strategy. By rearranging the positions of the data in different stages, memory access conflicts can be completely avoided. Third, this article employed the TF sharing strategy and the ROM partitioning strategy along with a conflict-free prime TF access method. In this way, it can support multiple BUs simultaneously using only one TF module, hence saving the hardware overhead of extra TF module.

III. ALGORITHM ANALYSIS AND IMPROVEMENT

A. Mixed-Radix FFT

The formula for the DFT is defined as shown in (1), where $x(n)$ and $X(k)$ are the input and output data series, respectively, of the system (neither the input nor output of intermediate stages). N is the size of the original DFT. W_N^{nk} is the n th coefficient, or TF, of the DFT. An FFT is formulated by decomposing the original DFT of a long series into cascaded DFTs of a short series. The procedure for solving the DFT of r -point decomposition is also called the radix- r butterfly operation

$$\begin{cases} X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk} \\ W_N^{nk} = \exp\left(-j \frac{2\pi nk}{N}\right). \end{cases} \quad (1)$$

A new mathematical operation $\{\}$ is defined in (2) that decomposes decimal data z into $\{z_{a1}, z_{a2}, \dots, z_{ai}, \dots, z_{at}\}$, where $z_{ai} \in [0, r_{ai})$. R_{ai} is the radix of the a_i th number and it also represents the radix of the a_i th FFT stage and t is the number of FFT stages. Mathematical operation $\{\}$ transforms decimal index to mixed-radix format. Accordingly, n and k can be expressed as shown in (3), while k_i is the new index of n_i after stage- i butterfly computation. The solution procedure of $x(k)$ needs several stages, since DFT has been decomposed into cascaded DFTs. Therefore, we redefine the format of the data indexes in the intermediate stages as well, as shown in (4), where K_i is the output data index of the i th stage. If $i = t$, then K_i is the same as k . In CFMBT, the DIT FFT algorithm is adopted, and the iterative process of the DIT FFT calculation can be described as shown in (5), where x_i is the output of the i th FFT stage. The W coefficients in the second line in (5) are called the interstage TFs of stage $i + 1$, meaning TFs between the two butterflies. The W coefficients in the first and last line in (5) are named the intrastage TFs, which means TFs in DFT of a short series

$$z = \{z_{a1}, z_{a2}, \dots, z_{at}\} = \sum_{j=1}^{t-1} \left(z_{a_j} * \prod_{i=j+1}^t r_{a_i} \right) + z_{a_t} \quad (2)$$

$$(a_1 \neq a_2 \neq a_3 \neq \dots \neq a_t \& a_1, a_2, a_3 \in [1, t]) \quad (2)$$

$$\begin{cases} n = \{n_1, n_2, \dots, n_i, n_{i+1}, n_{i+2}, \dots, n_t\} \\ k = \{k_1, k_2, \dots, k_i, k_{i+1}, k_{i+2}, \dots, k_t\} \end{cases} \quad (3)$$

$$K_i = \{k_1, k_2, \dots, k_i, n_{i+1}, n_{i+2}, \dots, n_t\} \quad (4)$$

$$\begin{cases} x_1(K_1) = \sum_{n=0}^{r_1-1} x(n) W_{r_1}^{n_1 k_1} \\ x'_i(K_i) = x_i(K_i) W_{r_1 * r_2}^{n_{i+1} * k_1} \\ \quad \times x_i(K_i) W_{\prod_{\alpha=1}^{i+1} r_{\alpha}}^{n_{i+1} * \left\{ \left(\sum_{\beta=2}^i \left(\prod_{\lambda=1}^{\beta-1} r_{\lambda} \right) * k_{\beta} \right) + k_1 \right\}}, \\ \quad (i = 1) \text{ or } (i > 1) \\ x_{i+1}(K_{i+1}) = \sum_{n_{i+1}=0}^{r_{i+1}-1} x'_i(K_i) W_{r_{i+1}}^{n_{i+1} * k_{i+1}}, \quad (i \neq 0). \end{cases} \quad (5)$$

Fig. 3(b)–(e) show the architectures of the radix-2/3/4/5 BUs based on the DIT FFT algorithm. Extra complex number multipliers are added in the BUs to achieve the multiplication of the interstage TFs. In Fig. 3, intrastage TFs have been design as constant coefficients, because intrastage TFs in

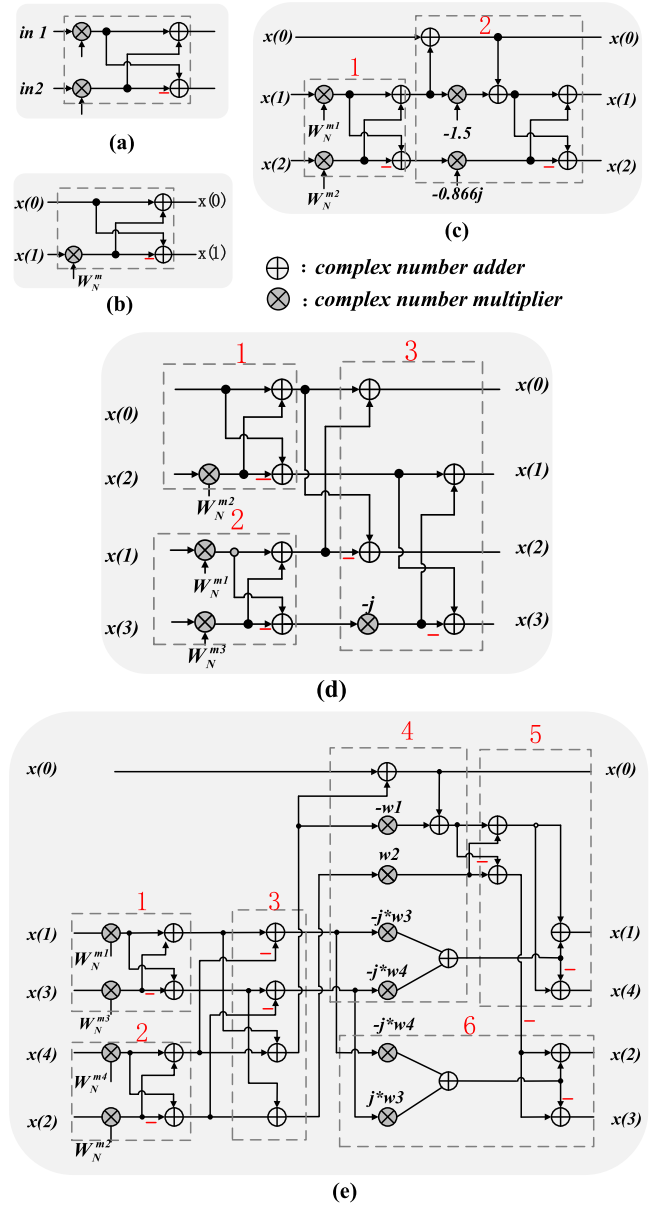


Fig. 3. BUs of radix-2/3/4/5. (a) General part, (b) radix-2, (c) radix-3, (d) radix-4, and (e) radix-5.

each radix are determined. Fig. 3(a) shows the part of the architecture referred to as the general structure. The general structure is extracted from the common parts of the radix-2/3/4/5 BU architectures. As we can see in Fig. 3, the general structure is present in the radix-2 BU, part 1 of the radix-3 BU, parts 1 and 2 of the radix-4 BU and parts 1 and 2 of the radix-5 BU. Therefore, defining this general structure as a PE can enhance the hardware resource reuse ratio. In addition, a fine-grained hardware partitioning strategy is implemented in the BUs. One complex number multiplier is divided into four real number multipliers and two real number adders, and one complex number adder is divided into two real number adders. As a result, one instance of the general structure is composed of eight real number adders and eight real number multipliers. Both the adders and multipliers are implemented with half-precision floating point arithmetic units as described in [34].

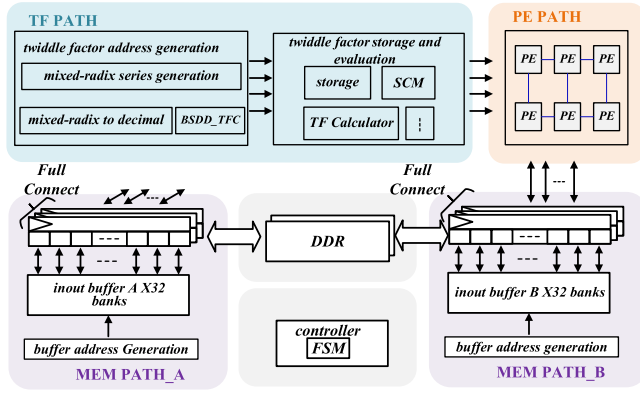


Fig. 4. Overview of the CFMBT architecture.

B. Bit Reordering Schedule

In the time-domain series of the N -point FFT, each data point corresponds to an index. As indicated in (4), we divide index K_{i-1} into a new mathematical format $\{k_1, k_2, \dots, k_{i-2}, k_{i-1}, n_i, \dots, n_t\}$ with n_i or $k_i \in [0, r_i)$, called a mixed-radix series, where r_i is the radix of the i th FFT stage. In this format, each data point has a unique $k_1, k_2, \dots, k_{i-1}, n_i, n_{i+1}, \dots, n_t$ combination. Equation (6) shows the data fetching sequence in the i th stage, where n_i or k_i can also be regarded as data of radix- r_i . I_i progressively increases from 0 to $N - 1$ and generates the corresponding $k_{i-1}, k_{i-2}, \dots, k_1, n_t, n_{t-1}, \dots, n_i$ values. Data can be fetched on the basis of the generated $k_{i-1}, k_{i-2}, \dots, k_1, n_t, n_{t-1}, \dots, n_i$ values since each data point has a unique $k_{i-1}, k_{i-2}, \dots, k_1, n_t, n_{t-1}, \dots, n_i$ combination

$$I_i = \left\{ \underbrace{k_{i-1}, k_{i-2}, \dots, k_1}_{i-1}, \underbrace{n_t, n_{t-1}, \dots, n_i}_{t-i+1} \right\} \quad (6)$$

value of TF in stage $-i$

$$= W_N^{n_i * \{k_{i-1}, k_{i-2}, \dots, k_1\} * \prod_{j=i+1}^t r_j} \quad (7)$$

$$N = \prod_{i=1}^t r_i (r_\alpha \geq r_\beta \text{ if } \alpha > \beta). \quad (8)$$

In (7), only changes in $n_i, k_{i-1}, k_{i-2}, \dots, k_1$ can affect the TF value. In (7), n_i also corresponds to different TF ports for the same BU, from port 0 to port $r_i - 1$. In (6), the values of $k_{i-1}, k_{i-2}, k_{i-3}, \dots, k_1$ change more slowly than those of $n_t, n_{t-1}, n_{t-2}, \dots, n_i$. Therefore, this schedule causes the TFs to change at the slowest speed, and the value of a TF can be shared by $\prod_{j=i+1}^t r_j$ BUs.

IV. HARDWARE ARCHITECTURE

A. Overview

Fig. 4 shows a system-level overview of the proposed CFMBT architecture. In accordance with the memory-based FFT algorithm, the architecture comprises three paths, where MEM PATH and TF PATH simultaneously provide the data and TFs, respectively, for PE PATH. Then, the result from PE PATH will be transferred back to MEM PATH. In MEM PATH, data are initially read from the double data rate (DDR) memory and written to buffer A. Then, the calculation of one FFT stage is started. Data are read from buffer A into the

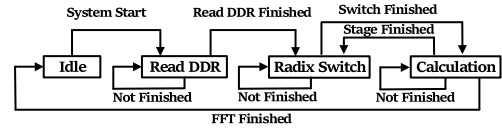


Fig. 5. State transition diagram of the controller.

PE array, and the calculation results are written from the PE array into buffer B. The address of the data in memory is given by the *buffer address generation* unit. In the next FFT stage, the results are read from buffer B, and after the PE calculations, they are written to buffer A. This iterative process is repeated until the calculation of all FFT stages is finished and the final result is obtained. In TF PATH, the interstage TFs are not directly stored in RAM. They are generated from several prime TFs instead. The *TF address generation* unit generates the address of a prime TF through the *mixed-radix series generation*, *mixed-radix to decimal* and *BSDD_TFC* units. Then, according to the generated addresses, in *TF storage and evaluation* unit, the desired prime TF values are fetched from a *storage* unit composed of RAM and RegFiles. After the *symmetric compression multiplexers (SCMs)* unit and some simple calculations in the *TF calculator*, the final value of the TF is calculated. In PE PATH, a 1×6 PE array is responsible for processing the butterfly computations. These six PEs are divided into several groups, each of which forms a particular type of BU in accordance with specified configuration information.

The *controller* is a finite state machine (FSM) that provides control signals to the other units. There are a total of four states: Idle, Read DDR, Radix Switch, and Calculation. Fig. 5 shows the state transition diagram. The CFMBT operation begins when a system start signal is received. In the Read DDR state, buffer A obtains data from the DDR memory. Then, the state transitions to Radix Switch, and the PE array is reconstituted as radix-2, radix-3, radix-4, or radix-5 BUs. The radix of the BUs is determined by the radix of the current FFT stage. After Radix Switch is finished, the controller transitions to the Calculation state. At each jump to the Calculation state, the PE array processes one of the multiple stages of the FFT calculation. In the Calculation state, data are read from buffer A (or buffer B) into the PE array, and interstage TFs are also read from the *TF storage and evaluation* unit into the PE array. The calculation results from the PE array are written from the PE array into buffer B (or buffer A). Upon completion, the system either returns to the Idle state or returns to the Radix Switch state, which transforms the PE array into BUs of another radix. If the FFT Finished signal is not true, then the Calculation state transitions to the Radix Switch state to start the calculation of the next FFT stage. The iterative process of transitioning from Calculation to Radix Switch and back to Calculation is repeated until all FFT stages have finished and the final result has been obtained. Then, the FFT Finished signal is true, and the state transitions to Idle.

B. PE Array

A reconfigurable PE array is proposed to achieve high throughput and a high hardware reuse rate. Fig. 6 shows the architecture of the PE array, which is composed of six unified PEs and a MUX matrix. The PEs are mainly responsible for the calculation functions of the PE array, while the MUX

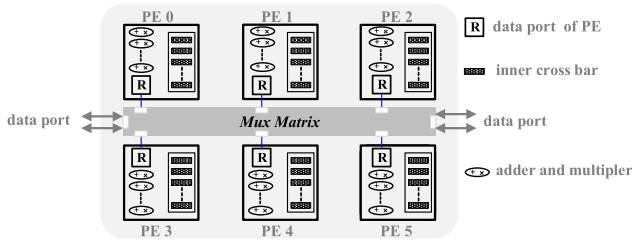


Fig. 6. Architecture of the PE array.

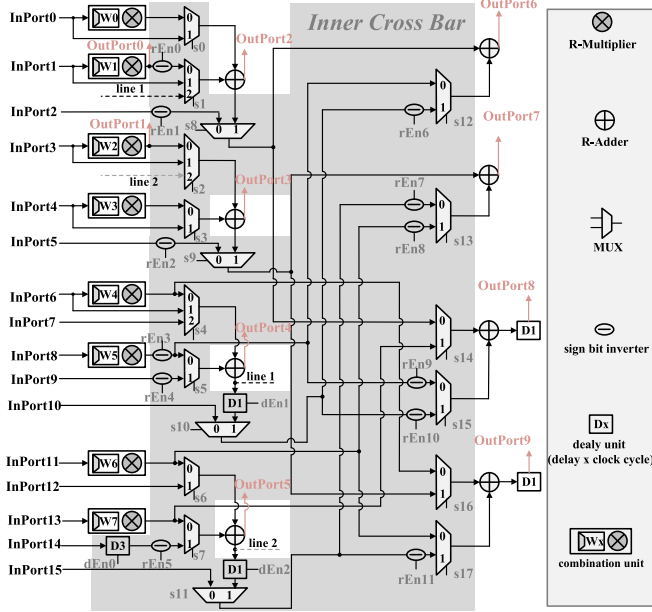


Fig. 7. Architecture of a PE.

matrix is mainly responsible for data transfer between PEs. Meanwhile, the MUX matrix provides an external access port for the PE array, through which external data from the buffer can be transferred into the PE array and the results from the PE array can be written to the buffer. Both the PEs and the MUX matrix are reconfigurable, which means that the PEs can be transformed to perform multiple calculation functions, while the MUX matrix can alter the connection relations between PEs or between the PE array and the buffer through different selection signals. There are four types of configurations for the PE array, corresponding to four array partitioning modes. For radix-2 calculations, each single PE can transform into a radix-2 BU, which means that the six PEs can process six butterfly computations simultaneously. For radix-3, the PE array can be reconfigured into three BUs, where each BU is formed by two PEs: PE0 and PE1, PE2 and PE3, and PE4 and PE5. For radix-4, PE0, PE1, and PE2 cooperate to act as one radix-4 BU, while PE3, PE4, and PE5 form another radix-4 BU. For radix-5, all six PEs cooperate to form one radix-5 BU. In this way, our PE array can be reconfigured for six radix-2 in parallel, three radix-3 in parallel, two radix-4 in parallel, and a radix-5 in one clock cycle.

As shown in Fig. 7, one PE is composed of real number adders (R -adders), combination units (containing real number multipliers, R -multipliers) and an inner cross bar. The R -adders and combination units are the computation modules of the PE, while the inner cross bar is responsible for

data transfer between computation modules. The numbers of R -adders and R -multipliers in the PE are both 8, as determined by the general structure proposed in Section III. Both the R -multipliers and R -adders process are implemented half-precision floating-point arithmetic units as described in [34]. Each combination unit is composed of a $W \times$ port, registers, MUXes, and R -multipliers. The $W \times$ port is responsible for accepting interstage TFs from TF PATH, as shown in Fig. 4, while the registers in PE store the intrastage TFs. The intrastage TFs depend only on the radix of the BUs (i.e., radix-2/3/4/5) instead of the variable number of FFT points. In our design, one or two registers storing intrastage TF in each combination unit is enough. The MUXes are responsible for choosing the intra- or inter-stage TFs, while the R -multipliers multiply the data by the TFs. The inner cross bar, which is marked in Fig. 7, is composed of delay units, MUXes and sign bit inverters. The delay units are applied to maintain the same data path delay from the input port to the output port by delaying data by one or three clock cycles; the enable signal dEn determines whether a delay unit delays the data passing through it. The MUXes are responsible for choosing different data paths in accordance with the selection signal s . Each sign bit inverter inverts the sign bit of the floating-point number passing through it when the corresponding enable signal rEn is true.

MUX matrix is a kind of cross bar which is composed of delay units, MUXes and sign bit inverters. To distinguish it from inner cross bar, we name it as MUX matrix. The data path in the MUX matrix can be altered in accordance with different selection signals. Thus, multiple PEs cooperate through the MUX matrix and to achieve the functions of various BUs. The MUX matrix also provides 24 16-bit input ports and 24 16-bit output ports, through which the buffer can access the PE array. The number of input-output ports is determined based on the maximum bandwidth of a PE. In the radix-2 case, each BU (or PE) receives two complex data points from the buffer and generates two complex data points to return to the buffer. Our PE array can be configured as six radix-2 BUs, meaning that 12 complex number data ports, that is, 24 16-bit real number data ports, are needed.

To describe the reconfiguration process of PE array in detail, we provided the mapping of radix-4 BU. For radix-4, the PE array can accommodate two radix-4 BUs. PE0, PE1, and PE2 cooperate to act as one radix-4 BU, while PE3, PE4, and PE5 form another radix-4 BU. For one radix-4 BU shown in Fig. 3(d), part 1, part 2, and part 3 of radix-4 are mapped on PE0, PE1, and PE2, respectively. The structure of part 1 in Fig. 3(d) requires one complex multipliers and two complex adders, occupying four real multipliers and six real adders. The lines among multipliers and adders in part 1 of Fig. 3(d) are realized by changing the data path through MUXes in PE0. In Fig. 3(d), data $x(0)$ do not require complex multiplication, while data $x(2)$ require complex multiplication. Thus, to keep data [i.e., $x(0)$ and $x(2)$] alignment in time, the delay unit in PE is enabled on the input path of $x(0)$, thereby performing one clock cycle delay for $x(0)$. In this way, the output result from the complex multiplier can be synchronized with delayed data $x(0)$, and then are sent to subsequent two complex adders in part 1. When the complex adder in part 1 performs subtraction, the sign bit inverters unit in PE is enabled, causing the sign bit of the subtraction to change. In the same way, the part 2 and part 3 in Fig. 3(d) are mapped onto PE1 and PE2. For radix-4 BU, MUX matrix in PE array realizes interconnections

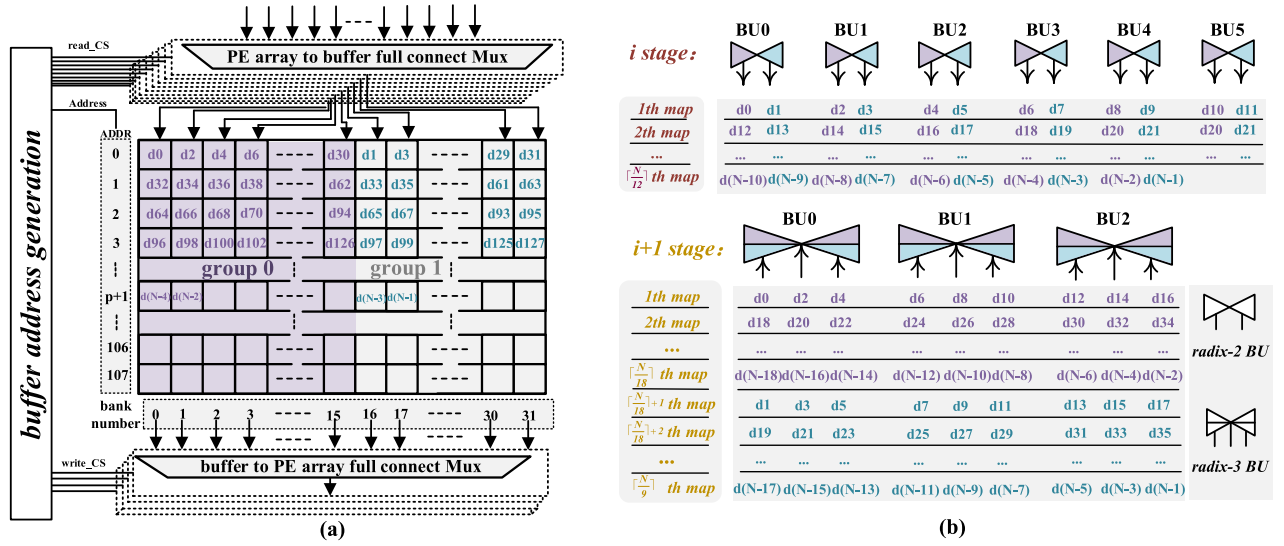


Fig. 8. Structure of buffer A/B. (a) Data organization in the buffer. (b) Data flow in BUs.

among part 1, part 2, and part 3 in Fig. 3(d). Besides, the MUX matrix is also connected with the data buffer, so that data can be read from the buffer, as well as writing results back to the buffer.

In PE array, the key innovation exists in the proposed reconfigurable PE and MUX matrix, where different BUs for radix2/3/4/5 can be mapped on the unified hardware architecture. A BU is divided into multiple parts, and each PE is responsible for a part of BUs operations. Such fine grain division of BU and mapping method on PE array is beneficial for achieving high hardware utilization in radix2/3/4/5.

C. Buffer and Conflict-Free DDMAS

Fig. 8(a) shows the detailed architecture of MEM PATH. There are three main parts: the *buffer*, the *buffer address generation* unit, and the *fully connected MUX* units. The *buffer* is responsible for storing the input-output data or intermediate results of the FFT calculation, while the *buffer address generation* unit provides bank selection for data buffer (BS_DB) signals and addresses for the buffer. Each *fully connected MUX* unit is responsible for choosing the PE array's input-output ports and choosing the buffer banks in accordance with the BS_DB signals. In our design, the hardware overhead *fully connected MUX* units is optimized, as introduced in the next paragraph. One buffer has 32 banks with a depth of 108, for a total of $32 \times 108 = 3456$ memory cells, which is sufficient to store up to 3240 FFT points. Each memory cell in a bank stores a 32-bit complex number, with the higher 16 bits for the real part and the lower 16 bits for the imaginary part. The *buffer address generation* unit is responsible for generating addresses and BS_DB signals in the *buffer address generation* unit are shown in Table II. The read_BS_DB, write_BS_DB, read_addr, and write_addr signals are the outputs of the *buffer address generation* unit. The read_BS_DB and write_BS_DB signals are the BS_DB signals, while the read_addr and write_addr signals are the addresses for the buffer. The symbol $\lfloor \cdot \rfloor$ in Table II denotes the rounding-down operation. t is the total number of FFT stages, and r_i is the radix corresponding to stage i . Data are dispatched

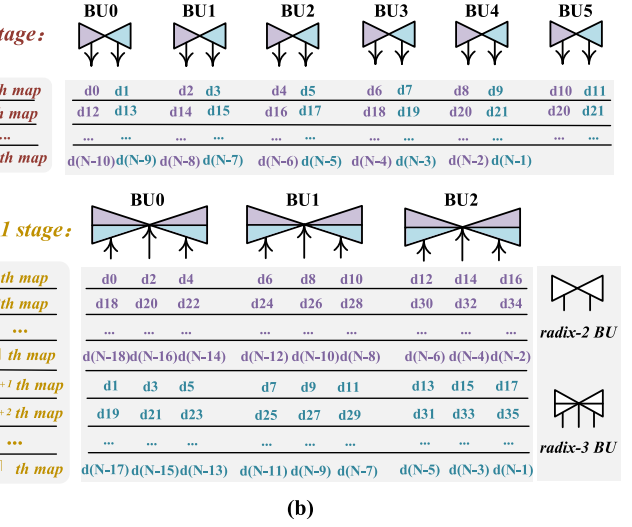


TABLE II
PSEUDOCODE OF THE DDMAS METHOD

Address and BS_DB generation strategies of DDMAS	
/*Read Strategy*/ output: read_BS_DB and read_addr	
/*calculate FFT stage by stage*/	
For($i=1; i \leq t; i++$) {	
$r_0 = r_i$	
/*parallelism means calculation parallelism of radix- r_i on PE array*/	
$parallelism(r_i) = 6$ ($r_i=2$) or 3 ($r_i=3$) or 2 ($r_i=4$) or 1 ($r_i=5$)	
/*in read process of the i -th stage, buffer is divided into r_{i-1} groups*/	
$gBank = \lfloor \cdot \rfloor$ /* how many banks per group*/	
$gSize =$ /* how much data one group stores*/	
For($j=0; j < r_{i-1}; j++$) { /*Read data in the order of the group*/	
$bank=0$, read_BS_DB=0, read_addr=0/* parameter initialization */	
For(count=0; count < gSize; count = count + $r_{i-1} \times parallelism(r_{i-1})$) {	
/*following loop is unrolled and mapped onto the PE array in parallel */	
For($k=0; k < parallelism(r_i); k++$) { /* k is the index of the BUs*/	
For($z=0; z < r_i; z++$) { /* z is the index of a BU's ports*/	
$mid = read_BS_DB$	
$BU(k,z) \Rightarrow read_BS_DB = (bank + r_i \times k + z) \% gBank + j \times gBank$	
If (read_BS_DB > mid) read_addr = read_addr + 1	
ELSE read_addr = read_addr	
$bank = (bank + r_i \times parallelism(r_i)) \% gBank$ }	
/*Write Strategy*/ output: write_BS_DB and write_addr	
For($i=1; i \leq t; i++$) {	
$parallelism(r_i) = 6$ ($r_i=2$) or 3 ($r_i=3$) or 2 ($r_i=4$) or 1 ($r_i=5$)	
/*in the write process of the i -th stage, the buffer is divided into r_i groups*/	
$gBank = \lfloor \cdot \rfloor$ /* how many banks per group*/	
$gSize =$ /* how much data one group stores*/	
For(count=0; count < gSize; count = count + $parallelism(r_i)$) {	
$bank=0$, write_BS_DB=0, write_addr=0/* parameter initialization */	
/*following loop is unrolled and mapped onto the PE array in parallel */	
For($k=0; k < parallelism(r_i); k++$) { /* k is the index of the BUs*/	
For($z=0; z < r_i; z++$) { /* z is the index of a BU's ports*/	
$mid = write_BS_DB$	
$BU(k,z) \Rightarrow write_BS_DB = (bank + k) \% gBank + gBank \times z$	
If (write_BS_DB > mid) write_addr = write_addr + 1	
ELSE write_addr = write_addr	
$bank = (bank + parallelism(r_i)) \% gBank$ }	

in accordance with the bit reordering schedule. Meanwhile, our DDMAS divides the banks into several groups in accordance with the radix corresponding to stage i or $i - 1$. Data are

written in different groups and read from the same group, exactly consistent with the data scheduling method used in the bit reordering schedule. Thus, our DDMAS is mathematically equivalent to the bit reordering schedule. In this way, memory access conflicts can be avoided. The *fully connected MUX* units are divided into a PE-array-to-buffer unit and a buffer-to-PE-array unit. The PE-array-to-buffer unit is responsible for choosing the operation results from the PE array and writing to the corresponding banks, while the buffer-to-PE-array unit is responsible for choosing data from the buffer to be passed to the corresponding PE array input ports. The PE-array-to-buffer fully connected MUX unit is composed of 32 24-1 MUXes with a data size of 16 bits. The dimensions of the 24-1 MUXes correspond to the 24 output ports of the PE array. Each bank has a 24-1 MUX to choose the output of the PE array; therefore, for 32 banks, 32 24-1 MUXes are needed. In the same way, the buffer-to-PE-array fully connected MUX unit is composed of 24 32-1 MUXes. The MUX dimensions of 32-1 are determined by the 32 banks, and the number of MUXes required 24 is determined by the 24 input ports of the PE array.

In our design, the number of banks is optimized from 60 to 32. This is because for the same number of memory cells, the fewer banks there are, the lower the hardware overhead of the *fully connected MUX* units will be. Therefore, we impose the constraint that $r_i \geq r_j$ if $i > j$. This constraint avoids the worst case, in which 60 banks would be needed. Equation (9) shows the minimum number of banks needed in the i th stage. This equation is deduced from Table II. According to Table II, to avoid the case in which BUs that are executed in parallel need to read from the same bank, the number of banks in a group needs to be larger than $r_i \times \text{parallelism}(r_i)$. In the worst case, a radix-2 stage follows a radix-5 stage and $r_{i-1} = 5$. In this case, $r_i \times \text{parallelism}(r_i) \times r_{i-1} = 2 \times 6 \times 5 = 60$ banks are needed. Our constraint that $r_i \geq r_j$ if $i > j$ avoids this case; under this constraint, the worst remaining case is the case in which a radix-4 stage follows a radix-4 stage. Therefore, the number of banks needed in the case that $r_i = 4$ and $r_{i-1} = 4$ is 32

$$\text{bank_number}_i = r_i \times \text{parallelism}(r_i) \times r_{i-1} \quad (i \geq 2). \quad (9)$$

To further explain our strategy, we take the process of writing data from a radix-2 stage to the buffer followed by fetching data for a radix-3 stage as an example, as shown in Fig. 8(b). Data shown in purple in Fig. 8(a) are group 0 data, while data in cyan are group 1 data. The arrows in Fig. 8(b) indicate the data flow for each BU port, and i and $i + 1$ denote consecutive FFT stages. In each stage, an N -point FFT is mapped onto the PE array. However, limited by the PE array's performance, the N point calculations of one stage need to be mapped several times. For radix-2, $\lceil (N/12) \rceil$ instances of mapping are needed, while for radix-3, $\lceil (N/9) \rceil$ mapping instances are needed. Here, $\lceil \cdot \rceil$ denotes the rounding-up operation.

1) *Write*: In stage i , the data come from the result of a radix-2 FFT calculation. Therefore, the 32 banks are divided into two groups called group 0 and group 1, each consisting of 16 banks. The banks in group 0 are marked in purple in Fig. 8(a), while the banks in group 1 are marked in gray. In the radix-2 case, our PE array supports six radix-2 BUs, and the butterfly operation produces two outputs; for example, the two outputs of BU0 are denoted by $d0$ and $d1$ in Fig. 8(b). The first outputs are stored in the first 16 banks in sequence,

as shown for $d0$, $d2$, $d4$, and $d6$ in Fig. 8(a) and (b). The second outputs are similarly stored in the last 16 banks in sequence, as illustrated for $d1$, $d3$, $d5$, and $d7$. In the abstract, if the first output of the first PE is stored in row q of bank (p) , then the first output of the second PE is stored in row $(q + \lfloor ((p+1)/16) \rfloor)$ of bank $(p+1)\%16$. In the same way, the second output of the first PE is stored in row q of bank $(p+16)$, while the second output of the second PE is stored in $(q + \lfloor ((p+1)/16) \rfloor)$ of bank $(p+1)\%16 + 16$, and so on.

2) *Read*: In stage $i + 1$, data are fetched from contiguous banks in a single group. The group division is determined by in *write* process in stage i . In this example case, the previous stage was a radix-2 FFT stage, and the banks were divided into two groups. Hence, the three input ports of the radix-3 BUs will initially fetch contiguous data, such as $d0$, $d2$, and $d4$ in Fig. 8(a) fetched by radix-3 BU0 in Fig. 8(b), from the first 16 banks. In the abstract, if the first input for the first radix-3 BU is fetched from row q of bank (p) , then the second input for the same radix-3 BU is fetched from row $(q + \lfloor ((p+1)/16) \rfloor)$ of bank $(p+1)\%16$, and the third input is fetched from row $(q + \lfloor ((p+2)/16) \rfloor)$ of bank $(p+2)\%16$. After all data have been taken out of the first 16 banks, data will be read from the next 16 banks in the same way, such as $d1$, $d3$, $d5$, $d7$, $d9$, and so on.

In buffer and DDMAS, key main improvement is a multibank buffer structure along with a DDMAS, achieving conflict-free access on data buffer. DDMAS is a hardware-friendly strategy and can avoid access conflict of data. In accordance with DDMAS, the multibank buffer is divided into different groups for different FFT stages. For a FFT stage, the output data from a BU is rearranged into different groups so as to meet the order requirement in bit reordering schedule. In this way, the input data of a BU in the next FFT stage is arranged at same group. In the next FFT stage, the input data of a BU can be fetched at the same clock cycle from different banks belonging to a group, thereby achieving conflict-free access on the multibank buffer.

D. Twiddle Factor Generation

Our *TF generation* unit implements a low-redundancy design without accuracy loss. Four optimization strategies are implemented: conflict-free prime TF access, a TF sharing strategy, a modified TF partitioning strategy, and a symmetric TF compression strategy. The conflict-free prime TF access method ensures that a single *TF generation* module can generate the entire set of interstage TFs needed by a BU rather than requiring a separate *TF generation* module to generate each TF. Moreover, with the TF sharing strategy, the *TF generation* unit can support all interstage TFs needed by the PE array. A modified TF partitioning strategy and a symmetric TF compression strategy are also implemented to further reduce the storage overhead. In the remainder of this section, the proposed redundancy-reducing storage methods will be introduced in Section IV-D1, and the hardware architecture of the *TF generation* unit is presented in Section IV-D2.

1) *Redundancy-Reducing Storage Methods*: A modified TF partitioning strategy is implemented to reduce the RAM scale needed to store the interstage TFs for FFT computations. The original TF partitioning strategy was proposed in [9]. Instead of storing all interstage TFs in memory, the TF partitioning strategy divides one ROM table into two smaller tables, as shown in (10). Thus, the storage scale can be reduced from N to $U + V$, where $W_U^{k_1}$ and $W_N^{k_2}$ are called the prime TFs.

However, in the method presented in [9], all possible $W_U^{k_1}$ and $W_N^{k_2}$ values are stored in ROM to support various numbers of FFT points. Although the work in [9] ensures that the prime TFs can be fetched directly for any number of FFT points without updating, the storage overhead is still large since the number of FFT points can vary widely. Based on the original TF partitioning strategy, we fix U to 12 (for $12 \times n$ -point mixed-radix FFTs) and 64 (for power-of-two FFTs) and store the results in RegFiles, while $W_N^{k_2}$ is stored in RAM. The RegFiles' data are fixed, while the RAM storing the $W_N^{k_2}$ values will be updated when the number of points for the FFT calculation changes. The RAM size requirement is reduced from the size needed to hold all $W_N^{k_2}$ values to only the size needed to hold the $W_N^{k_2}$ values in the maximum N case. The process of updating occurs in the first and second FFT stages, and does not incur any additional time overhead. In the first stage, the FFT data are multiplied by a real number one, so the TF does not need to be fetched from the *TF generation* unit. Section IV-C mentions that the calculation priorities of different radices are assigned as radix-2 and radix-4 in the power-of-two case or radix-2, radix-3, radix-4, and radix-5 in the mixed-radix case. Therefore, in the second stage, the interstage TFs are W_6^k and W_{12}^k (in the mixed-radix case) or W_{16}^k and W_8^k (in the power-of-two case). All of them can be fetched from RegFiles. Considering that the maximum TR of the PE array is 12 points in each clock cycle, the total time cost of the first and second FFT stages is $N/12 + N/12 = N/6$ clock cycles. When our RAM consists of seven banks and updates seven prime TFs each clock cycle, $N/12$ interstage TFs can be updated in $N/(7 \times 12)$ clock cycles, which is fewer than $N/6$ clock cycles. Then, in the next stage, $W_N^{k_2}$ has been updated and can be fetched directly

$$\begin{cases} W_N^k = W_N^{V_{k_1+k_2}} = W_U^{k_1} \times W_N^{k_2} \\ N = U \times V, \quad k_1 \in [0, U), \quad k_2 \in [0, V). \end{cases} \quad (10)$$

A symmetric TF compression strategy, which is mentioned in [35], is applied to reduce the storage overhead for power-of-two FFTs. With this strategy, we need to store only 1/8 of the TFs in memory, and the others can be obtained via a symmetry algorithm. Therefore, when U in $W_U^{k_1}$ is fixed to 64, the RegFiles need to store only $W_{64}^0 - W_{64}^8$ rather than $W_{64}^0 - W_{64}^{63}$. Suppose that $W_{64}^x = a + b * i$ ($0 \leq x \leq 8$) and $W_{64}^y = c + d * i$ ($0 \leq y \leq 63$). W_{64}^x is stored in Reg 0-8 of the RegFiles, where Reg x ($0 \leq x \leq 8$) in RegFiles stores the value of W_{64}^x . The values of c and d can be obtained by exchanging a as the imaginary part and b as the real part or changing the sign bits of a and b . This process is known as decompression and is hardware friendly. Fig. 9 shows the architecture of an SCM, which is responsible for evaluating $W_{64}^9 - W_{64}^{63}$. s_0 is responsible for the exchange between the real part and the imaginary part. When its value is 0, this exchange will not occur. s_1 and s_2 are responsible for sign bit inversion. When the value of s_1 or s_2 is one, sign bit inversion will occur. From Reg 0-8 in the RegFiles, we can obtain the mapping table between RegFiles 0-8 and $W_{64}^0 - W_{64}^{63}$. This mapping function is expressed as $x = f(y)$. Table III shows the configuration context for generating W_{64}^y , where No.reg refers to the label of Reg 0-8 in the RegFiles.

Our conflict-free prime TF access method utilizes the properties of prime numbers. One BU usually needs a set of interstage TFs $W_N^{k_1}$, such as W_N^b , W_N^{2b} , W_N^{3b} , and W_N^{4b} , where the k_1 value increases in equal intervals. For example, for W_N^b ,

TABLE III
MAPPING RELATIONS AND CONFIGURATION INFORMATION OF SCMS

y^1	0-8	9-16	17-24	25-32	33-40	41-48	49-56	57-63
x^2 or No.reg	y	[16-y]		[32-y]		[48-y]		64-y
s0	0	1		0		1		0
s1s2		00		10		11		01

$$1. W_{64}^x = a + b * i \quad 2. W_{64}^y = c + d * i$$

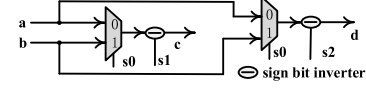


Fig. 9. Symmetric compression multiplexer (SCM).

W_N^{2b} , W_N^{3b} , and W_N^{4b} , the interval is b . Through the properties of prime numbers, a single memory access can be used to fetch four TFs at equal intervals without conflict. In other works, four memory accesses are needed to avoid memory access conflicts. However, in our work, the number of storage banks is set to a prime number to effectively reduce memory access conflicts. Our finding is as follows: suppose that the initial number of banks is a and that we need to fetch c TFs ($c \leq a$) separated by an interval b . If a and b are mutually prime, then those c TFs will come from different banks. If $b = z * a$ ($z \in 1, 2, 3, \dots$), then all of those c TFs will come from the bank 0. Therefore, when a is a prime number, a memory access conflict will occur only in the case of $b = z * a$. In the $b = z * a$ case, $c - 1$ extra banks storing the same data as bank 0 are needed to avoid memory access conflicts. In (10), $W_U^{k_1}$ is stored in the RegFiles, and $W_N^{k_2}$ is stored in RAM. We set a to the prime number 7 and c to 4 while adding $c - 1$ extra banks, all of which store W_N^{z*a} ($z \in 1, 2, 3, \dots$). Therefore, the final overhead in terms of the number of banks is $a + c - 1$. When $a = 7$ and $c = 4$, the number of banks of RAM needed is ten. Through this method, c prime TFs separated by an interval b can be fetched without memory access conflict.

The TF sharing strategy enables butterfly operations with the same interstage TFs to be mapped to the PE array at the same time. Therefore, several BUs can share the same interstage TFs, which can reduce the number of *TF generation* units needed. According to (6), I_i increases from 0 to $N - 1$ and generates the corresponding $k_{i-1}, k_{i-2}, \dots, k_1, n_i, n_{i-1}, \dots, n_1$ values. According to (7), the TF values depend on the values of n_i and k_1 to k_{i-1} . For data with the same values of n_i and k_1 to k_{i-1} , the same TF can be used. Multiple TFs which are needed by one BU are named as a set of TFs. n_i corresponds to different TF ports for the same BU, from port 0 to port $r_i - 1$. Hence, a set of TFs are shared by multiple BUs whose TFs have the same k_1 to k_{i-1} value. Taking the second stage of FFT as an example, I_i is $\{k_1, n_i, n_{i-1}, n_{i-2}, \dots, n_2\}$ according to (6) and (7), and corresponding TF value is presented in (11). Since k_1 changes more slowly than $n_i, n_{i-1}, n_{i-2}, \dots, n_2$, we assume k_1 as 1 in this example. In the progress of calculation, according to I_i , radix- r_2 BU0 fetch r_2 data and these r_2 data which are multiplied with $W_{N_2}^{0*k_1}, W_{N_2}^{1*k_1}, \dots, W_{N_2}^{n_2*k_1}$. When k_1 is 1, the TF value is $W_{N_2}^0, W_{N_2}^1, \dots, W_{N_2}^{n_2}$. Then I_i increases and $n_i, n_{i-1}, n_{i-2}, \dots, n_3$ changes. However, TF value is only decided by n_2 and k_2 , therefore BU1 fetch r_2 data, and these r_2 data is multiplied with $W_{N_2}^0, W_{N_2}^1, \dots, W_{N_2}^{n_2}$ as well. As a result, BU0 and BU1 can share the same TF value

$$\text{value of TF in stage } - 2 = W_{N_2}^{n_2*k_1} \left(N_2 = N / \prod_{j=3}^t r_j \right). \quad (11)$$

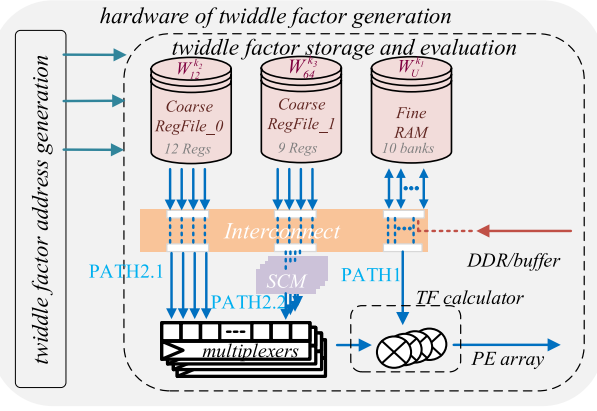


Fig. 10. Architecture of the TF generation unit.

2) *Hardware Architecture of the TF Generation:* Fig. 10 shows the hardware architecture of the TF generation unit, which is composed of a TF address generation unit and a TF storage and evaluation unit. The TF address generation unit is responsible for generating data depth for TF (DDTF) and bank selection for TF (BS_TF) signals to be sent to the TF storage and evaluation unit, which receives these signals and then generates corresponding interstage TFs. First, the TF address generation unit generates BS_TF and DDTF signals for Fine RAM and Coarse RegFiles, which store the compressed TFs: Fine RAM for $W_N^{k_2}$ and Coarse RegFiles for $W_U^{k_1}$. The TFs that are stored in the Fine RAM are called fine TFs, and those that are stored in the Coarse RegFiles are called coarse TFs. Both the Fine RAM and the Coarse RegFiles consist of multiple banks or registers, which means that they can accept four addresses and fetch four data entries simultaneously. Then, the fine and coarse TFs are fetched and transmitted to complex number multipliers. After this step, the TFs from PATH 1 can be sent to the complex multipliers directly, whereas the data from PATH 2.1 or PATH 2.2 must be subjected to further selection. Coarse RegFile_0 is for the interstage TFs of a mixed-radix FFT, and Coarse RegFile_1 is for a power-of-two FFT. Hence, complex number multiplexers are used to select data from PATH 2.1 or PATH 2.2 in accordance with the number of points of the FFT. In PATH 2.2, the data from Coarse RegFile_1 need to be decompressed through SCMs and then passed to the multipliers. After the data from PATH 1 and PATH 2 have been multiplied in the complex multipliers, a set of interstage TFs required by a BU can be generated and sent to the PE array. The data in Coarse RegFile_0 and Coarse RegFile_1 are fixed regardless of the number of points of the FFT. However, in the Fine RAM, the data need to be updated from the DDR memory or buffer when the number of FFT points changes.

Fig. 11(a) shows the storage methods of Coarse RegFile_0 and Coarse RegFile_1. Coarse RegFile_0 is for mixed-radix FFTs and has 12 registers, from Reg 0 to Reg 11. Each register stores a 32-bit TF in complex number format, with the higher 16 bits for the real part and the lower 16 bits for the imaginary part. These 12 registers store TF values from W_{12}^0 to W_{12}^{11} , corresponding to $W_U^{k_1}$ ($U = 12$ in our design) in (10). Coarse RegFile_1 has nine registers, from Reg 0 to Reg 8, storing W_{64}^0 – W_{64}^8 . On the basis of the symmetric TF compression strategy, Coarse RegFile_1 can generate interstage TFs from W_{64}^0 to W_{64}^{63} through SCMs. In contrast, Coarse RegFile_1 is designed for power-of-two FFTs. Fig. 11(b) shows the

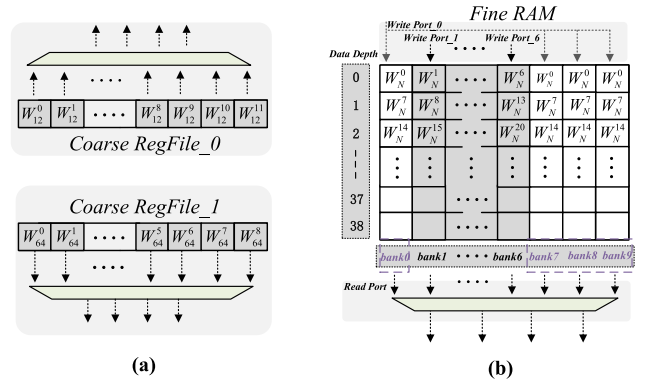


Fig. 11. Hardware and data organization architectures of (a) Coarse RegFiles and (b) Fine RAM.

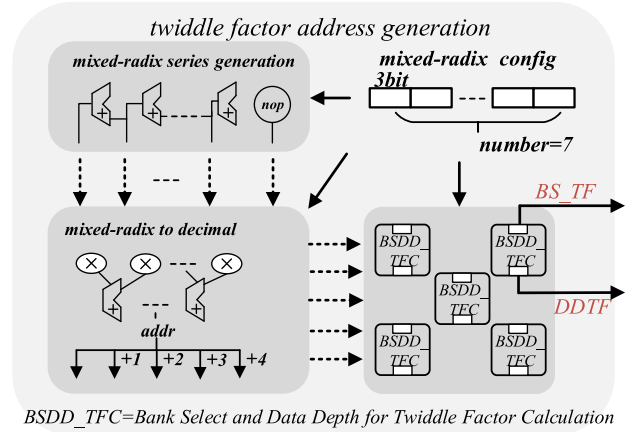


Fig. 12. Hardware architecture of the TF address generation.

architecture of the Fine RAM, which is responsible for storing the $W_N^{k_2}$ values appearing in (10). It comprises ten banks with a depth of 39. Each data entry in the Fine RAM has a width of 32 bits. Banks 7–9 store the same data as bank 0; the purpose of this value copying operation is to resolve memory access conflicts. Prime TFs are stored in bank 0 to bank 6 in sequence, from W_N^0 to $W_N^{((N/12)-1)}$, as shown in Fig. 11(b)

$$k = n_i * \{k_{i-1}, k_{i-2}, \dots, k_1\} * \prod_{j=i+1}^t r_j \quad (12)$$

$$\begin{cases} \alpha = N/12 \text{ (mixed-radix) or } N/64 \text{ (power of two FFT)} \\ \beta = k \% \alpha \end{cases} \quad (13)$$

$$\begin{cases} \text{BS_TF}_{\text{FineRAM}} = \beta \% 7, & (\beta \% 7 \neq 0) \\ \text{BS_TF}_{\text{FineRAM}} = n_j + 7, & (\beta \% 7 = 0 \ \& \ n_j \neq 0) \\ \text{BS_TF}_{\text{FineRAM}} = n_j, & (\beta \% 7 = 0 \ \& \ n_j = 0) \\ \text{DDTF}_{\text{FineRAM}} = \lfloor \beta / 7 \rfloor \end{cases} \quad (14)$$

$$\begin{cases} \text{BS_TF}_{\text{Coarse RegFiles}} = \lfloor k / \alpha \rfloor, & \text{(mixed-radix)} \\ \text{BS_TF}_{\text{Coarse RegFiles}} = f(\lfloor k / \alpha \rfloor), & \text{(power of two FFT)} \\ \text{DDTF}_{\text{Coarse RegFiles}} = 0. \end{cases} \quad (15)$$

Fig. 12 shows the architecture of the TF address generation unit. It is composed of a mixed-radix series generation unit, a mixed-radix to decimal unit, and a Bank Selection and Data

Depth for TF calculation (BSDD_TFC) unit. The *mixed-radix series generation* unit is used to generate mixed-radix series $n_{i-1}, n_{i-2}, \dots, n_{i-1}, n_i, n_{i+1}, \dots, n_t$ in accordance with (6) in the bit-reordering schedule. As mentioned in the description of the TF sharing strategy, the mixed-radix series generation method used in the bit reordering schedule takes advantage of the shareability of the TFs. As a result, memory access conflicts are decreased by reducing the number of accesses to memory. The *mixed-radix series generation* unit performs a function similar to that of a counter, and the counting result is expressed in mixed-radix format. The *mixed-radix to decimal* unit processes the mixed-radix series from the *mixed-radix series generation* unit and outputs the decimal value of k in W_N^k . The processing method of the *mixed-radix to decimal* unit is shown in (12). The BSDD_TFC unit process data in both mixed-radix format and decimal format to produce BS_TF and DDTF signals. The calculation process performed in the BSDD_TFC unit is expressed in (13)–(15). For a mixed-radix FFT, the address generation method for Coarse RegFile_0 is shown in (15). Given that each TF in Coarse RegFile_0 is stored in an independent register, multiple interstage TFs can be fetched in a conflict-free manner. The address generation method for the Fine RAM is slightly complex, as shown in (14). In (14), we set the initial number of banks to 7, as determined by the conflict-free prime TF access method, while the final number of banks is 10. For power-of-two FFTs, there is some difference compared with the mixed-radix case. Through the symmetric TF compression strategy, $W_{64}^0 - W_{64}^{63}$ can be obtained from $W_{64}^0 - W_{64}^8$. When W_{64}^x or W_{64}^y needs to be fetched, the corresponding bank in Coarse RegFile_1 will be chosen, and the data will be transferred to the SCM. The address calculation formula for power-of-two FFTs is shown in (14) and (15), where $f(y)$ is the mapping function of the SCM, as shown in Table III. If $N \leq 64$, TFs will not be fetched from the Fine RAM. If $N > 64$, then the address calculation formula for the Fine RAM is the same as (14).

In TF generation, three improvement strategies (i.e., conflict-free prime TF access, a TF sharing strategy, a modified TF partitioning strategy) are proposed to achieve a low-redundancy design without accuracy loss. Before improvement, each TF port of each BU needs its own *TF generation* unit, and each BU also needs a *TF generation* unit. With the help of the conflict-free prime TF access method and the TF sharing strategy, a single *TF address generation* unit can support the whole BUs mapped on PE array. Also, using these two strategies, the need for storage size is compressed from $6N$ to N at most. Moreover, by the aid of the modified TF partitioning strategy and the symmetric TF compression strategy, the requirement of storage size can be further reduced. For mixed-radix FFTs, the storage size can be further compressed from N to $5N/42 + 12$ (i.e., $N/12 \times 10/7 + 12$); while for power-of-two FFTs, the number of storage units can be reduced from N to $5N/224 + 9$ (i.e., $N/64 \times 10/7 + 9$) at most.

V. EXPERIMENTAL EVALUATION

A. Architecture Analysis and Comparison

Our proposed CFMBT system architecture was synthesized with Design Compiler in semiconductor manufacturing international corporation (SMIC) 40-nm CMOS technology. CFMBT supports up to 3240 points and 61 different FFT sizes modes while supporting radix-2, radix-3, radix-4, and radix-5 operations in half-precision floating point format.

To fairly compare the pros and cons of different FFT architectures, we consider two concepts: the number of compute units (CUs) and the computation efficiency (CE). The number of CUs is defined as the sum of the numbers of adders and multipliers. Generally, the more CUs there are, the higher the peak performance that is achieved. Floating adders and floating multipliers occupy similar hardware overhead, as described in [34]. Therefore, we do not distinguish between adders and multipliers as CUs, and the total number of CUs is what we are concerned with. The CE indicates how many FFT points each CU delivers in every clock cycle and is defined as shown in (16). Therefore, the CE reflects the performance of an architecture in the case of a given number of CUs

$$CE = \frac{\text{Throughput Rate}}{\text{number of computer unit(CU)}} \times 1000. \quad (16)$$

Several memory-based FFT architectures are considered for comparison, as listed in Table IV, including two memory-based DFT processors [28], a modified radix-4 processor [36], a highly flexible mixed-radix FFT processor [2], a radix-2^k processor [12], and a radix-4 processor [37]. In other works, non-power-of-two FFTs are also called DFTs, and power-of-two FFTs are still called FFTs. Therefore, to distinguish between non-power-of-two and power-of-two FFTs, we adopt these expressions in the following comparisons.

First, the TR and computation resources are evaluated, which are correlated with the data scheduling strategy and number of PEs. Compared with an FFT processor with two computation cores (a DFT core and an FFT core) [28], CFMBT achieves a 40% higher TR while saving approximately 25% CUs in the DFT core. In contrast, in the FFT core, the processor presented in [28] saves 30% CUs compared with our CFMBT architecture. However, CFMBT still achieves a 40% higher TR. In addition, for an input time series, its size is fixed. The time series will be sent to only one of the computation cores. Therefore, the DFT and FFT cores in [28] may not be utilized at the same time, resulting in an inability to reach the theoretical peak performance. Compared with [2], CFMBT incurs only 1/4 of the CU overhead while achieving 36%–71% of the TR. This is because the complex number adders and multipliers are divided into fine grain (i.e., real number operators), thereby enhancing the utilization of the digital signal processors (DSPs) in CFMBT. Compared with the radix-4 processor [36], CFMBT achieves a $1.1 \times -1.9 \times$ speedup in TR. However, our CU resource usage is $3 \times$ that of [36]. The low CU overhead of the processor presented in [36] is a benefit of its single-radix design. In addition, the high radix (i.e., radix-4) implemented in [36] is conducive to enhancing the peak performance in the case of given hardware resources. However, the architecture in [36] supports only FFTs of radix-4. Therefore, it can achieve peak performance only under the condition of power-of-four FFTs, which may result in a large number of invalid computations due to complement 0 in the case of a non-power-of-four FFT. Therefore, for flexible input sizes (e.g., the 1536-point FFT used in 5G), the work in [36] exhibits significant performance degradation. Compared with CFMBT, the work in [12] achieves a $2.5 \times$ speed-up in TR, which is due to its much higher radix (i.e., radix-16) and abundant CU resources ($3.3 \times$ those of CFMBT) in the FFT core. However, the limitation on the input size in [12] is even more severe than that in [36]. Therefore, it is not appropriate for use in OFDM systems with high flexibility requirements. Meanwhile, the TR in [37] is 43%

TABLE IV
ARCHITECTURE COMPARISON WITH OTHER MEMORY-BASED PROCESSORS

Architecture	Proposed	[28]TVLSI 2017		[2]TVLSI 2019	[36]TCAS-I 2019	[12]TVLSI 2020	[37]TCAS-II 2022
TR	1.4R ¹ (1536 points) 1.4R (2048 points)	1R	1R	1.95R (1200 points) 3.8R (2048 points)	1.3R (256 points) 0.73R (2048 points)	3.5R (2048 points)	2R (2048 points)
FFT points	2 ^a 3 ^b 5 ^c or 2 ^a	2 ^a 3 ^b 5 ^c	8 ^a	2 ^a 3 ^b 5 ^c or 2 ^a	4 ^a	16 ^a	4 ^a
RA	48	72	32	358	22	192	80
RM	48	56	24	45	12	128	29
CU ²	96	128	56	403	34	320	109
CE ³	14.5 (1536 points) 14.5 (2048 points)	7.8	17.8	4.8 (1200 points) 9.4 (2048 points)	38.2 (256 points) 21.4 (2048 points)	10.9 (2048 points)	18.3 (2048 points)
Memory Size	2N(single port)	2N(dual-port)	3N(dual-port)	2N(single port)	2N(single port)	N(single port)	N(dual-port)
Data Format	floating-point	fixed-point	fixed-point	fixed-point	fixed-point	fixed-point	fixed-point

1. TR=Throughput Rate

2. R=points per clock cycle

3. CU(Compute Unit)=RA+RM

4. CE(Computational Efficiency)=1000×Throughput Rate/CU

higher than that of CFMBT. This high performance in [37] can be attributed to its single-radix (i.e., radix-4) BUs and its architecture's clever use of the odd conjugate symmetry of 2^k -point FFTs. However, the effectiveness of this optimization strategy is limited for mixed-radix FFTs.

Second, under conditions of high flexibility, CFMBT achieves a high CE and low data memory overhead. A higher CE means a higher TR of each CU. On the one hand, compared with the FFT cores in the mixed-radix FFT processors of [2] and [28], CFMBT achieves $1.9\times$ to $3\times$ speed-ups in the TR of each CU. Compared with the high-radix processor of [12], CFMBT achieves a 33% improvement in CE while supporting input series of more diverse sizes. Compared with the single-radix processors of [36] and [37], our CE is lower for larger Fourier transform sizes, such as 2048 points. However, only radix-4 is supported in these ASIC cores, which means that their applicability scenarios are limited. On the other hand, CFMBT is also advantageous in terms of data memory overhead. According to [30], the area of single-port memory is approximately 56% less than that of dual-port memory. Thus, for the same storage capacity, implementing single-port memory enables greater area savings. CFMBT implements a *single-port* data memory with a size of $2N$. The hardware overhead is the same as that in [2] and is smaller than those of the two DFT processors in [28], whose memory sizes are $2N$ and $3N$ with *dual ports*. Compared with [36], CFMBT maintains the same memory consumption while additionally supporting non-power-of-four FFTs. Although our memory size is larger than that in [12], considering flexibility and CE, the memory scheme in [12] is more suitable for systems with abundant computation resources that require only power-of-two FFTs. Finally, the work in [37] requires a dual-port memory with a size of N , the area of which is similar to that of a $2N$ single-port memory. Therefore, the area overhead of the data memory is similar to that of CFMBT.

B. Performance Comparison

Table V lists performance comparisons between our CFMBT architecture and other state-of-the-art works (e.g., memory-based, SDF-based, and fully unrolled architectures). Overall, CFMBT attains the largest FFT size modes while achieving higher data accuracy with the help of 16-bit floating point operations. The maximum FFT mode of 61 can meet the needs of more application scenarios, while the maximum

mode among other mixed-radix cores is 54. Meanwhile, our CFMBT FFT core also accomplishes higher throughput and area efficiency than other mixed-radix FFT processors [2], [11], [28], [38] in the DFT case. CFMBT even shows an advantage in throughput compared with a 128-point FFT processor [37]. Notably, the FFT processors in Table V are implemented in different CMOS technologies. Therefore, we adopt the SAR introduced in [9], defined as shown in (17), to evaluate the performance per unit area in a more equitable way. This metric represents how many points each gate delivers in every clock cycle. A high SAR of an FFT processor indicates that it achieves a high area efficiency

$$\text{Speed - Area Ratio (SAR)} = \frac{\text{Throughput}}{\text{Gate Count}} \quad (17)$$

$$\text{Norm Area} = \frac{\text{Area}}{(L_{\min}/40 \text{ nm})^2} \quad (18)$$

The CFMBT architecture provides a throughput of approximately 650M points/s with 365k logic gates, achieving an SAR value of 1719 for FFTs and DFTs in a typical case. Compared with [28], CFMBT achieves $1.9\times$ to $4.7\times$ higher SAR values with a $5.3\times$ speed-up in throughput. Compared with [38], CFMBT achieves a $5.81\times$ increase in the SAR with a $5.89\times$ speed-up in throughput. The high SAR of our proposed architecture mainly benefits from two reasons. One reason is that we implemented a reconfigurable PE array. Xia et al. [28] designs multiple BUs for radix 2 and radix3/4/5, respectively. In this way, if radix 2 is used in FFTs one stage, the rest radix3/4/5 BUs are in idle. As a result, it caused low SAR of [28]. On the contrast, the proposed CFMBT architecture can reuse the CUs in reconfigurable PEs, so that different radices' butterfly computations can be mapped on the same PE array. Therefore, the CFMBT architecture can reduce the hardware resource in idle, saving plenty of hardware overhead. Another reason is that our clock frequency is much higher than [28] and [38]. High clock frequency is benefit from the dedicatedly designed multistage pipeline, so as to reducing combinational logical delay. Besides, more advanced CMOS technology is beneficial for achieving higher clock frequency. The proposed CFMBT architecture was implemented using SMIC 40 nm technology, while [28] and [38] were implemented in SMIC 55 nm and TSMC 90nm technologies, respectively. Compared with [2] in the DFT case, our architecture achieves $1.34\times$ and $1.84\times$ improvements in throughput and SAR value,

TABLE V
PERFORMANCE COMPARISON WITH OTHER FFT PROCESSORS

Architecture	Proposed	[28]TVLSI 2017		[11]TCAS-I 2018	[2]TVLSI 2019	[38]JCSAI 2019	[39]TCAS-II 2021				[37]TCAS-II 2022
FFT Points	12-3240 ($2^{13}3^{15}$) 16-2048 (2^n)	12-1296 ($2^{13}3^{15}$)	128-2048 (2^n)	4-4096 ($2^{13}3^{15}$)	12-2400 ($2^{13}3^{15}$) 16-4096(2^n)	4-2048 ($2^{13}3^{15}$)	32	64	128	256	128
Number of Model	61	35	6	48	54	36	1	1	1	1	1 [*]
Frequency (MHz)	483	122.88		526.32	250	111	750	500	500	500	100
Throughput (M symbols/s)	654 (1536 points) 654 (2048 points)	122.88		526.32	487 (1200 points) 950 (2048 points)	111	1333	1777	3200	6400	200
Technology (nm)	SMIC 40nm	SMIC 55 nm		TSMC 40nm	SMIC 65nm	TSMC 90nm	TSMC 65nm				NOC ^{*2} 45nm
Gate Count	365k	340k	136k	445k	522k	374k	196k	285k	509k	1205k	66k
SAR ^{*1}	1719 (1536 points) 1719 (2048 points)	361	903	1183	933 (1200 points) 1808 (2048 points)	296	6801	6235	6286	5311	3030
Method	Memory-Based	Memory-Based		SDF	Memory-Based	Memory-Based	Fully-Unrolled				SDF
Wordlength (bit)	16bits	16bits		14bits	14bits	16bits	9/10 bits	9/11 bits			-
Data format	float-point	fixed-point	fixed-point	fixed-point	fixed-point	fixed-point	fixed-point				fixed-point
Area (Norm.Area)	0.836mm ² (0.836)	1.063mm ² (0.562)	0.615mm ² (0.325)	0.414mm ² (0.414)	1.464mm ² (0.554)	1.416mm ² (0.280)	0.22mm ² (0.083)	0.32mm ² (0.121)	0.91mm ² (0.345)	2.03mm ² (0.769)	0.063mm ² (0.050)

*1 SAR=Speed-Area Ratio

*2 NOC=NangateOpenCell

respectively. Although in the FFT case, our throughput is 30% lower than that of the processor in [2], our SAR value is similar to that in [2], which means that our area efficiency is nearly the same. However, compared with [37] and [39], the throughput and SAR values of CFMBT are lower. One reason is that the processors in [37] and [39] are designed for a small number of certain FFT points. In these two works, one ASIC core can only support FFTs of a certain size and certain radix, which lacks flexibility. On the contrast, CFMBT can support much richer radices and FFT points, thereby requiring extra hardware resources. Besides, due to the configurability, the utilizations of CUs (i.e., multiplier and adder) in PEs cannot reach 100% for each radix, thereby leading to performance degradation of CFMBT. Another reason is that both [37] and [39] employs fixed-point CUs using the data width of 9/11 bits, which can significantly save hardware resources. As a contrast, CFMBT implement 16 bits' width of floating-point data which introduced high hardware overhead. Even in this case, CFMBT is still competitive in area efficiency, which is due to our highly reconfigurable architecture's ability to achieve full reuse of its unified hardware resources. The SAR in our work is only 44% lower than that in [37]. However, in both of these designs (see [37] and [39]), once the FFT core has been determined, neither the FFT size nor the radix can be changed, which is not flexible enough for OFDM systems. Generally, our throughput and SAR are superior to those of other mixed-radix FFT processors, and they are also competitive when compared with those of specific-size FFT processors.

A summary of the area cost and comparison with other works are also given in Table V. According to [28], the normalized area is defined as (18), where $L_{\min}$ is the CMOS technology used in other works. Using (18), all the competitors can be normalized to the same technology (i.e., 40 nm). The CFMBT architecture occupies 0.836 mm² core area, while the normalized areas of other works are at the range from 0.050 to 0.769 mm². The comparison of normalized area shows that CFMBT is not superior to other works. This is mainly due to the floating-point numbers used in CFMBT, while all of the competitors employ fixed-point numbers. Besides, more

memory banks designed in CFMBT will lead to greater area overhead as well.

VI. CONCLUSION

This article proposed a unified dynamically reconfigurable FFT architecture that implements a DIT-FFT-based PE array with a flexible conflict-free memory structure. Compared to the state-of-the-art designs, our architecture supports 61 different FFT size modes with higher CE. An innovative TF sharing structure is adopted to minimize storage redundancy and support multiple butterfly units. The FFT architecture has been implemented in SMIC 40-nm CMOS technology with a working frequency of 483 MHz, performing at 654 MS/s for 2048-point and 1536-point FFTs.

REFERENCES

- [1] V. V. Vasilica, "FFT implementation on FPGA for 5G networks," M.S. thesis, Dept. Electr. Eng., Linköping Univ., Linköping, Sweden, 2019.
- [2] S. Liu and D. Liu, "A high-flexible low-latency memory-based FFT processor for 4G, WLAN, and future 5G," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 3, pp. 511–523, Mar. 2019.
- [3] C. Ingemarsson, P. Källström, F. Qureshi, and O. Gustafsson, "Efficient FPGA mapping of pipeline SDF FFT cores," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 9, pp. 2486–2497, Sep. 2017.
- [4] M. Garrido, R. Andersson, F. Qureshi, and O. Gustafsson, "Multiplier-less unity-gain SDF FFTs," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 24, no. 9, pp. 3003–3007, Sep. 2016.
- [5] M. Wang and Z. Li, "A hybrid SDC/SDF architecture for area and power minimization of floating-point FFT computations," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2016, pp. 2170–2173.
- [6] S. K. Bashar, "An efficient approach to the computation of fast Fourier transform(FFT) by Radix-3 algorithm," in *Proc. ICIEV*, May 2013, pp. 1–5.
- [7] S.-Y. Peng, K.-T. Shr, C.-M. Chen, and Y.-H. Huang, "Energy-efficient 128~2048/1536-point FFT processor with resource block mapping for 3GPP-LTE system," in *Proc. ICGCS*, 2010, pp. 14–17.
- [8] C. Yu and M.-H. Yen, "Area-efficient 128- to 2048/1536-point pipeline FFT processor for LTE and mobile WiMAX systems," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 9, pp. 1793–1800, Sep. 2015.
- [9] X.-Y. Shih, Y.-Q. Liu, and H.-R. Chou, "48-mode reconfigurable design of SDF FFT hardware architecture using Radix-3² and Radix-2³ design approaches," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 64, no. 6, pp. 1456–1467, Jun. 2017.

- [10] X.-Y. Shih, H.-R. Chou, and Y.-Q. Liu, "VLSI design and implementation of reconfigurable 46-mode combined-Radix-based FFT hardware architecture for 3GPP-LTE applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 1, pp. 118–129, Jan. 2018.
- [11] X.-Y. Shih, H.-R. Chou, and Y.-Q. Liu, "Design and implementation of flexible and reconfigurable SDF-based FFT chip architecture with changeable-Radix processing elements," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 11, pp. 3942–3955, Nov. 2018.
- [12] J. Wang, S. Li, and X. Li, "Scheduling of data access for the Radix-2^k FFT processor using single-port memory," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 7, pp. 1676–1689, Jul. 2020.
- [13] F. Qureshi, M. Ali, and J. Takala, "Multiplierless reconfigurable processing element for mixed Radix-2/3/4/5 FFTs," in *Proc. IEEE Int. Workshop Signal Process. Syst. (SiPS)*, Oct. 2017, pp. 1–6.
- [14] F. Qureshi, M. Garrido, and O. Gustafsson, "Unified architecture for 2, 3, 4, 5, and 7-point DFTs based on Winograd Fourier transform algorithm," *Electron. Lett.*, vol. 49, no. 5, pp. 348–349, Feb. 2013.
- [15] Q. Zhang, C. Liu, P. Zhang, Y. Chen, and J. Du, "Improved unified architecture for 3, 5, and 7-point Winograd Fourier transform algorithm," in *Proc. IEEE 3rd IMCEC*, Oct. 2019, pp. 1545–1548.
- [16] M. A. Shami, M. A. Tajammul, and A. Hemani, "Configurable FFT processor using dynamically reconfigurable resource arrays," *J. Signal Process. Syst.*, vol. 91, no. 5, pp. 459–473, May 2019.
- [17] V. Lalwani and S. Munir, "Area efficient VLSI architecture for reversible Radix-2 FFT algorithm using folding technique and reversible gate," in *Proc. 3rd ICOEI*, Apr. 2019, pp. 1429–1433.
- [18] F. Ke, O. Chen, Y. Wang, and N. Yoshikawa, "Demonstration of a 47.8 GHz high-speed FFT processor using single-flux-quantum technology," *IEEE Trans. Appl. Supercond.*, vol. 31, no. 5, pp. 1–5, Aug. 2021.
- [19] H. Chuppala, S. Chinthalapalli, V. Aditi, and R. K. Karunavathi, "Modified Cooley–Tukey algorithm for implementation of integrated serial FFT/IFFT processor in half-duplex OFDM systems," in *Proc. IEEE ICPCSI*, Sep. 2017, pp. 2328–2331.
- [20] E. J. Kim, J. H. Lee, and M. H. Sunwoo, "Novel shared multiplier scheduling scheme for area-efficient FFT/IFFT processors," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 9, pp. 1689–1699, Sep. 2015.
- [21] J. Wang, C. Xiong, K. Zhang, and J. Wei, "A mixed-decimation MDF architecture for Radix-2^k parallel FFT," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 24, no. 1, pp. 67–78, Jan. 2016.
- [22] H. Abdoli, H. Nikmehr, N. Movahedinia, and F. de Dinechin, "Improving energy efficiency of OFDM using adaptive precision reconfigurable FFT," *Circuits, Syst., Signal Process.*, vol. 36, no. 7, pp. 2742–2766, Jul. 2017.
- [23] M. Dali, A. Guessoum, R. M. Gibson, A. Amira, and N. Ramzan, "Efficient FPGA implementation of high-throughput mixed radix multi-path delay commutator FFT processor for MIMO-OFDM," *Adv. Electr. Comput. Eng.*, vol. 17, no. 1, pp. 27–38, Feb. 2017.
- [24] S.-J. Huang and S.-G. Chen, "A high-throughput Radix-16 FFT processor with parallel and normal input/output ordering for IEEE 802.15.3c systems," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 59, no. 8, pp. 1752–1765, Aug. 2012.
- [25] A. X. Glittas, M. Sellathurai, and G. Lakshminarayanan, "A normal I/O order Radix-2 FFT architecture to process twin data streams for MIMO," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 24, no. 6, pp. 2402–2406, Jun. 2016.
- [26] P. Bansal, B. S. Dhaliwal, and S. S. Gill, "Memory-efficient Radix-2 FFT processor using CORDIC algorithm," in *Proc. IEEE ICGCCEE*, Mar. 2014, pp. 1–5.
- [27] S. Y. Park and Y. J. Yu, "Fixed-point analysis and parameter selections of MSR-CORDIC with applications to FFT designs," *IEEE Trans. Signal Process.*, vol. 60, no. 12, pp. 6245–6256, Dec. 2012.
- [28] K.-F. Xia, B. Wu, T. Xiong, and T.-C. Ye, "A memory-based FFT processor design with generalized efficient conflict-free address schemes," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 6, pp. 1919–1929, Jun. 2017.
- [29] P.-C. Jui, C.-L. Wey, and M.-T. Shiue, "Low-cost parallel FFT processors with conflict-free ROM-based twiddle factor generator for DVB-T2 applications," in *Proc. IEEE 56th MWSCAS*, Aug. 2013, pp. 1003–1006.
- [30] H.-F. Luo, Y.-J. Liu, and M.-D. Shieh, "Efficient memory-addressing algorithms for FFT processor design," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 10, pp. 2162–2172, Oct. 2015.
- [31] X.-Y. Shih and H.-R. Chou, "A 2-D grouping FIFO based hardware architecture for supporting 36-mode hybrid-radix FFT design in 3GPP-LTE systems," in *Proc. IEEE 6th GCCE*, Oct. 2017, pp. 1–2.
- [32] S. J. A. C. Williams et al., "Roofline: An insightful visual performance model for floating-point programs and multicore," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [33] Y.-W. Lin, H.-Y. Liu, and C.-Y. Lee, "A 1-GS/s FFT/IFFT processor for UWB applications," *IEEE J. Solid-State Circuits*, vol. 40, no. 8, pp. 1726–1735, Aug. 2005.
- [34] C. Yang, S. Xiang, J. Wang, and L. Liang, "A high performance and full utilization hardware implementation of floating point arithmetic units," in *Proc. 28th ICECS*, Nov. 2021, pp. 1–4.
- [35] M. Hasan and T. Arslan, "Scheme for reducing size of coefficient memory in FFT processor," *Electron. Lett.*, vol. 38, no. 4, pp. 163–164, Feb. 2002.
- [36] Y. Tian, Y. Hei, Z. Liu, Q. Shen, Z. Di, and T. Chen, "A modified signal flow graph and corresponding conflict-free strategy for memory-based FFT processor design," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 66, no. 1, pp. 106–110, Jan. 2019.
- [37] C. Eleftheriadis and G. Karakostas, "Energy-efficient fast Fourier transform for real-valued applications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 5, pp. 2458–2462, May 2022.
- [38] X.-Y. Shih, "VLSI architecture of 36-mode reconfigurable FFT hardware chip with newly-developed 2D-FIFO arrangement structure," in *Proc. 6th Int. Conf. Syst. Informat. (ICSIAI)*, Nov. 2019, pp. 999–1003.
- [39] S. H. Mirfarshbafan, S. Taner, and C. Studer, "SMUL-FFT: A streaming multiplierless fast Fourier transform," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 5, pp. 1715–1719, May 2021.